



Tuyển tập Bài tập Thực hành: Các Kỹ thuật Xử lý Lưới Hình chữ nhật (2D Grid Techniques)

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)
Biên soạn: Được hỗ trợ và soạn thảo bởi Agentic AI tool
Bản quyền © 2026 Duc-Minh Vu. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

Danh mục Bài tập Phân cấp (Problem List)

STT	Tên bài toán	Nền tảng & ID	Cấp độ	Kỹ thuật cốt lõi
1	Counting Rooms	CSES 1192	● Cơ bản	Đếm thành phần liên thông trên lưới bằng DFS/BFS (Flood Fill)
2	Labyrinth	CSES 1193	● Cơ bản	BFS tìm đường đi ngắn nhất trên lưới và truy vết ngược hướng đi
3	Forest Queries	CSES 1652	● Cơ bản	Mảng cộng dồn 2D (2D Prefix Sums) truy vấn số cây trong $O(1)$
4	Grid Paths	CSES 1638	● Cơ bản	Quy hoạch động lưới $N \times N$ có chướng ngại vật $O(N^2)$
5	Chessboard and Queens	CSES 1624	● Cơ bản	Quay lui đặt 8 quân hậu, kiểm tra đường chéo chính $r - c$ và phụ $r + c$
6	Monsters	CSES 1194	● Trung cấp	Loang đa nguồn (Multi-Source BFS) kết hợp so sánh mốc thời gian
7	Maximal Rectangle	LeetCode 85 / CSES	● Trung cấp	Biến đổi lưới nhị phân thành biểu đồ cột + Monotonic Stack $O(R \cdot C)$
8	Maximum Submatrix Sum	Classical 2D Kadane	● Trung cấp	Nén cột kết hợp Kadane 1D tìm chữ nhật con tổng lớn nhất $O(R^2 \cdot C)$
9	0-1 BFS on Grid	AtCoder ABC 176 D	● Trung cấp	Deque 0-1 BFS: đi bộ chi phí 0, phép thuật dịch chuyển chi phí 1
10	Count Submatrices with All Ones	LeetCode 1504	● Trung cấp	Đếm số hình chữ nhật con toàn 1 bằng chiều cao cột và Stack $O(R \cdot C)$
11	Knight's Moves Shortest Path	Classical / VNOJ	● Cơ bản	BFS quân mã với vector 8 hướng <code>kx[8], ky[8]</code>
12	Bipartite Chessboard Covering	Codeforces / ICPC	● Trung cấp	Tô màu nhị phân $(r + c) \pmod 2$, phủ Domino trên lưới bị khuyết ô

STT	Tên bài toán	Nền tảng & ID	Cấp độ	Kỹ thuật cốt lõi
13	Sparse Grid Maze	AtCoder ARC	● Nâng cao	Rời rạc hóa và nén tọa độ lưới $10^9 \times 10^9$ có chương ngại vật thưa
14	Laser Bouncing in Grid	Codeforces 793C	● Nâng cao	Tia laser phản xạ trên lưới gương, mô phỏng chu kỳ và hình học
15	Escape from Fire Grid	ICPC Regional Mid	● Nâng cao	Đám cháy đa nguồn lan truyền theo thời gian $T(r, c)$ kết hợp Dijkstra
16	Rectangular Cutting / DP	CSES 1744	● Trung cấp	Quy hoạch động chia cắt hình chữ nhật $A \times B$ thành ít hình vuông nhất

● PHẦN 1: CẤP ĐỘ CƠ BẢN (RATING 800 — 1200)

Bài 1: CSES 1192 — Counting Rooms

- **Đề bài:** Cho bản đồ tòa nhà kích thước $n \times m$ gồm các ô sàn `.` và tường `#`. Các ô sàn kề nhau (chung cạnh) thuộc cùng một phòng. Đếm tổng số phòng.
- **Phân tích & Gợi ý:**
- Đây là bài toán tìm số thành phần liên thông trên đồ thị lưới.
- Duyệt qua từng ô (r, c) : nếu là `.` và chưa được ghé thăm, tăng số phòng lên 1 và gọi `dfs(r, c)` (hoặc `bfs`) để đánh dấu toàn bộ các ô thuộc cùng phòng đó.
- Sử dụng mảng hướng 4 phía `dr[4] = {-1, 0, 1, 0}`, `dc[4] = {0, 1, 0, -1}`.
- **Độ phức tạp:** $O(n \times m)$ thời gian và bộ nhớ.

Bài 2: CSES 1193 — Labyrinth

- **Đề bài:** Tìm đường đi ngắn nhất từ vị trí bắt đầu `A` đến vị trí kết thúc `B` trên mê cung $n \times m$. Nếu có đường đi, in ra độ dài và chuỗi các bước di chuyển (`U`, `D`, `L`, `R`).
- **Phân tích & Gợi ý:**
- Vì đồ thị lưới có trọng số cạnh không đổi ($= 1$), thuật toán **BFS** đảm bảo tìm ra đường đi ngắn nhất.
- Để truy vết: lưu mảng `parent_dir[r][c]` ghi lại ký tự hướng (`U`, `D`, `L`, `R`) đã dẫn tới ô (r, c) .
- Khi BFS chạm tới `B`, lần ngược từ `B` về `A` theo `parent_dir` rồi đảo ngược chuỗi kết quả.
- **Độ phức tạp:** $O(n \times m)$.

Bài 3: CSES 1652 — Forest Queries

- **Đề bài:** Cho một khu rừng kích thước $n \times n$ gồm các ô trống `.` và cây `*`. Có q truy vấn, mỗi truy vấn cho hình chữ nhật $[y_1, x_1]$ đến $[y_2, x_2]$, hỏi có bao nhiêu cây bên trong.
- **Phân tích & Gợi ý:**
- Xây dựng mảng cộng dồn 2D `pref[r][c]`:
$$pref[r][c] = pref[r-1][c] + pref[r][c-1] - pref[r-1][c-1] + (grid[r][c] == '*' \pmod{10^9 + 7})$$
- Trả lời mỗi truy vấn trong $O(1)$:

$$ans = pref[y_2][x_2] - pref[y_1-1][x_2] - pref[y_2][x_1-1] + pref[y_1-1][x_1-1]$$

- **Độ phức tạp:** $O(n^2)$ tiền xử lý, $O(1)$ cho mỗi truy vấn.

Bài 4: CSES 1638 — Grid Paths

- **Đề bài:** Cho bảng $n \times n$ có một số ô bẫy `*`. Bắt đầu tại $(1, 1)$, bạn chỉ có thể đi sang phải hoặc đi xuống dưới. Đếm số đường đi hợp lệ đến $(n, n) \pmod{10^9 + 7}$.
- **Phân tích & Gợi ý:**
- Quy hoạch động trạng thái: $DP[r][c]$ là số cách đi đến ô (r, c) .
- Nếu $grid[r][c] == '*'$: $DP[r][c] = 0$.
- Ngược lại:

$$DP[r][c] = (DP[r-1][c] + DP[r][c-1]) \pmod{10^9 + 7}$$

- Cơ sở: $DP[1][1] = 1$ (nếu ô $(1, 1)$ không phải là bẫy).
- **Độ phức tạp:** $O(n^2)$ thời gian, có thể tối ưu bộ nhớ còn $O(n)$ với DP cuộn mảng 1D.

● PHẦN 2: CẤP ĐỘ TRUNG CẤP (RATING 1300 — 1600)

Bài 6: CSES 1194 — Monsters

- **Đề bài:** Bạn bắt đầu tại ô `A` trên mê cung. Có nhiều quái vật tại các ô `M`. Cả bạn và quái vật đều di chuyển 1 bước mỗi giây. Nếu quái vật đến một ô cùng lúc hoặc trước bạn, bạn bị bắt. Tìm xem bạn có thể thoát ra biên mê cung an toàn không và in đường đi.
- **Phân tích & Gợi ý:**
- **Giai đoạn 1 (Loang Đa Nguồn cho Quái vật):**
 - Đẩy tất cả tọa độ các ô `M` vào cùng một hàng đợi BFS.
 - Tính mảng `monster_dist[r][c]`: thời gian sớm nhất một con quái vật bất kỳ có thể chạm tới ô (r, c) .
- **Giai đoạn 2 (BFS cho Bạn):**
 - Chạy BFS từ `A` để tính `my_dist[r][c]`.
 - Bạn chỉ được phép bước vào ô (nr, nc) nếu `my_dist[nr][nc] < monster_dist[nr][nc]`.
 - Nếu chạm tới bất kỳ ô nào nằm trên biên mê cung ($r = 0, n - 1$ hoặc $c = 0, m - 1$), ta đã thoát thành công! Truy vết đường đi.
- **Độ phức tạp:** $O(n \times m)$.

Bài 7: LeetCode 85 — Maximal Rectangle

- **Đề bài:** Cho ma trận nhị phân $R \times C$ gồm các ký tự `'0'` và `'1'`. Tìm diện tích hình chữ nhật lớn nhất chỉ chứa toàn số `'1'`.
- **Phân tích & Gợi ý:**
- Duyệt qua từng hàng r từ 0 đến $R - 1$.
- Duy trì mảng chiều cao các cột $H[c]$: nếu $grid[r][c] == '1'$ thì $H[c] += 1$, ngược lại $H[c] = 0$.
- Tại mỗi hàng, áp dụng thuật toán tìm **Hình chữ nhật lớn nhất trong Biểu đồ Cột** bằng Ngăn xếp đơn điệu (Monotonic Stack) trong $O(C)$.
- Cập nhật diện tích cực đại toàn cục.
- **Độ phức tạp:** $O(R \times C)$ thời gian, $O(C)$ bộ nhớ phụ.

Bài 9: AtCoder ABC 176 D — Wizard in Maze

- **Đề bài:** Bạn ở mê cung $H \times W$. Có hai cách di chuyển:
- Đi bộ trực giao sang 4 ô kề nhau (chi phí 0).
- Dùng phép thuật dịch chuyển tới bất kỳ ô nào trong hình vuông 5×5 tâm tại ô hiện tại (chi phí 1).
- Tìm chi phí phép thuật nhỏ nhất để đi từ điểm xuất phát tới đích.
- **Phân tích & Gợi ý:**
- Vì chi phí di chuyển chỉ có hai giá trị 0 và 1, đây là bài toán kinh điển của **0-1 BFS**.
- Dùng `std::deque<pair<int, int>>` và mảng khoảng cách `dist[r][c] = INF`:
 - 4 nước đi trực giao chi phí 0: nếu tối ưu hơn, gán `dist = cur` và đẩy vào `push_front`.
 - Các ô trong phạm vi 5×5 chi phí 1: nếu tối ưu hơn, gán `dist = cur + 1` và đẩy vào `push_back`.
- **Độ phức tạp:** $O(H \times W \times 25) = O(H \times W)$ thời gian.

● PHẦN 3: CẤP ĐỘ NÂNG CAO (RATING 1700 — 2000)

Bài 13: Sparse Grid Maze

- **Đề bài:** Mê cung có kích thước $10^9 \times 10^9$. Bắt đầu tại $(1, 1)$, đích tại $(10^9, 10^9)$. Có $K \leq 1000$ đoạn tường nằm ngang hoặc thẳng đứng. Tìm đường đi ngắn nhất hoặc kiểm tra tính liên thông.
- **Phân tích & Gợi ý:**
- Kích thước lưới 10^9 không thể mô phỏng trực tiếp.
- Áp dụng kỹ thuật **Nén Tọa độ Lưới Thưa (Coordinate Compression on 2D Grid)**:
 - Thu thập tất cả các mốc tọa độ x và y của các đầu mút đoạn tường, cùng các mốc $x - 1, x + 1$ và $y - 1, y + 1$.
 - Rời rạc hóa trục X và Y . Lưới mới chỉ có kích thước tối đa $\approx 4000 \times 4000$.
 - Đánh dấu các đoạn tường trên lưới đã nén.
 - Khoảng cách giữa hai ô nén kề nhau bằng hiệu khoảng cách thực sự trên tọa độ gốc.
 - Chạy thuật toán Dijkstra trên lưới nén để tìm đường đi ngắn nhất.
- **Độ phức tạp:** $O(K^2 \log K)$.



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agent AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.